

Esercitazione - Spring Boot REST API Producer/Consumer

Obiettivo

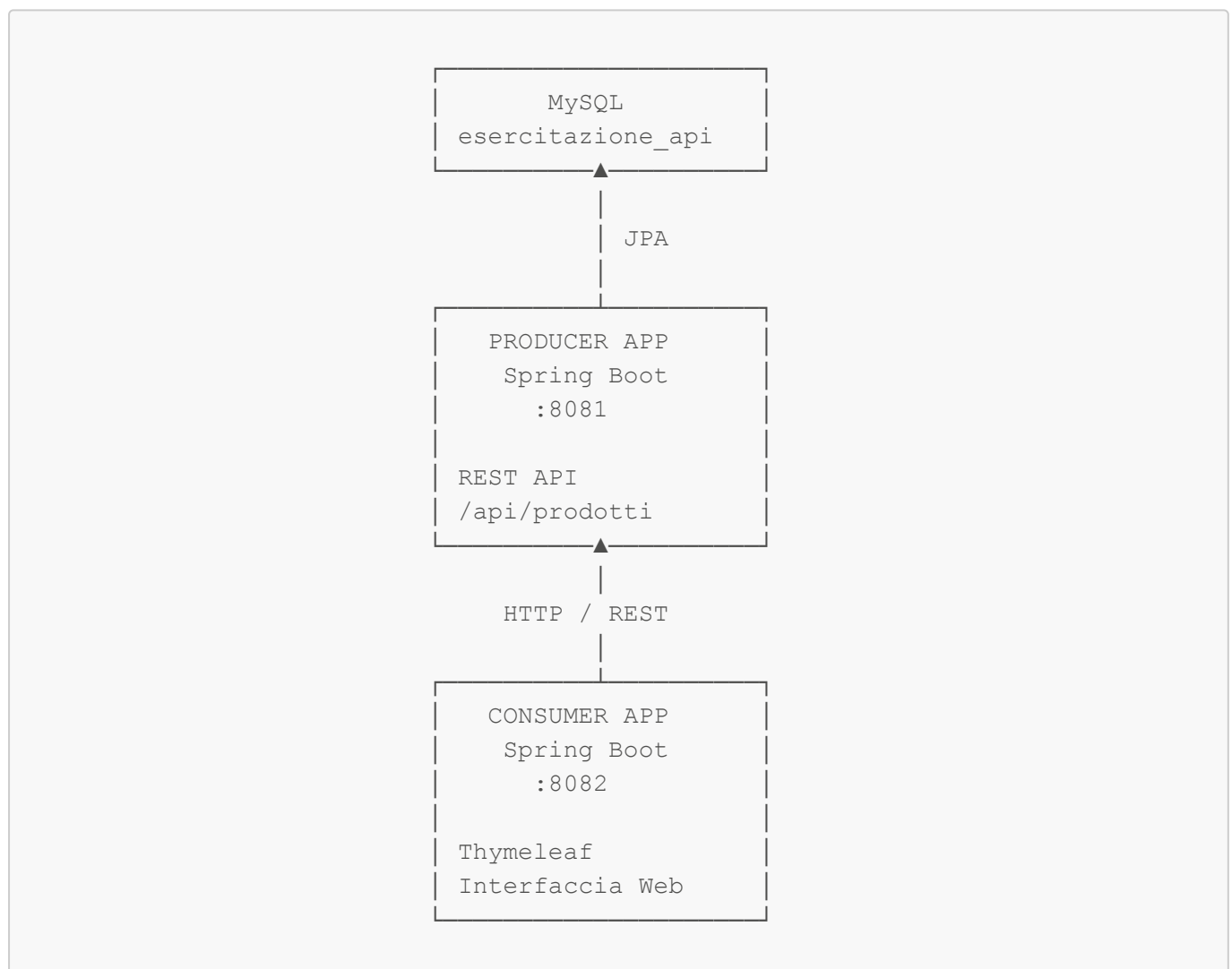
Realizzare un sistema composto da **due applicazioni Java indipendenti**, entrambe sviluppate con Spring Boot:

1. una **Producer Application**, responsabile della gestione dei prodotti e dell'esposizione di una REST API;
2. una **Consumer Application**, responsabile del consumo della REST API e della visualizzazione dei dati attraverso un'interfaccia web.

La Producer Application deve utilizzare un database MySQL per la persistenza dei dati.

La Consumer Application **non deve accedere direttamente al database**: tutti i dati devono essere recuperati attraverso le REST API esposte dalla Producer Application.

Architettura generale



Tecnologie

Il progetto deve essere realizzato utilizzando:

- **Java 21**
- **Spring Boot 3**
- **Spring Web**
- **Spring Data JPA**
- **Hibernate**
- **MySQL**
- **Thymeleaf**
- **Lombok**

Per il client HTTP è possibile utilizzare una delle seguenti soluzioni:

- `RestClient`
- `WebClient`
- `RestTemplate`

Per un progetto basato su Spring Boot 3 è consigliato utilizzare **RestClient**.

Database

Il database deve chiamarsi:

```
esercitazione_api
```

La Producer Application deve gestire una tabella denominata:

```
prodotti
```

Struttura della tabella

Campo	Tipo	Descrizione
<code>id</code>	BIGINT	Identificativo univoco
<code>nome</code>	VARCHAR(200)	Nome del prodotto
<code>descrizione</code>	TEXT	Descrizione del prodotto
<code>prezzo</code>	DECIMAL(10,2)	Prezzo
<code>categoria</code>	VARCHAR(100)	Categoria

Campo	Tipo	Descrizione
quantita	INT	Quantità disponibile
data_creazione	TIMESTAMP	Data di creazione

L'identificativo deve essere generato automaticamente dal database/JPA.

Creazione del database

Creare il database MySQL con:

```
CREATE DATABASE esercitazione_api  
CHARACTER SET utf8mb4  
COLLATE utf8mb4_unicode_ci;
```

Se necessario:

```
USE esercitazione_api;
```

La tabella può essere creata automaticamente da Hibernate utilizzando:

```
spring.jpa.hibernate.ddl-auto=update
```

In alternativa è possibile creare manualmente la tabella.

Script SQL completo

È possibile utilizzare il seguente script per creare database, tabella e dati iniziali.

```
CREATE DATABASE IF NOT EXISTS esercitazione_api  
CHARACTER SET utf8mb4  
COLLATE utf8mb4_unicode_ci;  
  
USE esercitazione_api;  
  
DROP TABLE IF EXISTS prodotti;  
  
CREATE TABLE prodotti (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(200) NOT NULL,  
    descrizione TEXT,
```

```

prezzo DECIMAL(10,2) NOT NULL,
categoria VARCHAR(100) NOT NULL,
quantita INT NOT NULL DEFAULT 0,
data_creazione TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

```

Dati di prova

Caricare nella tabella almeno i seguenti prodotti:

```

INSERT INTO prodotti
(nome, descrizione, prezzo, categoria, quantita, data_creazione)
VALUES
(
    'Laptop Pro 15',
    'Notebook professionale con processore Intel Core i7, 16 GB di RAM e
    SSD da 512 GB.',
    1299.90,
    'Informatica',
    15,
    '2026-09-01 09:00:00'
),
(
    'Laptop Air 13',
    'Notebook compatto e leggero con 8 GB di RAM e SSD da 256 GB.',
    899.00,
    'Informatica',
    22,
    '2026-09-02 10:15:00'
),
(
    'Monitor 27 4K',
    'Monitor professionale da 27 pollici con risoluzione 4K UHD.',
    449.90,
    'Informatica',
    12,
    '2026-09-03 11:30:00'
),
(
    'Tastiera Meccanica RGB',
    'Tastiera meccanica con retroilluminazione RGB e switch
    programmabili.',
    89.90,
    'Accessori',
    35,
    '2026-09-04 14:00:00'
),
(
    'Mouse Wireless Pro',

```

```

        'Mouse wireless ergonomico con sensore ad alta precisione.',
        59.90,
        'Accessori',
        48,
        '2026-09-05 09:45:00'
    ),
    (
        'Cuffie Bluetooth',
        'Cuffie wireless con cancellazione attiva del rumore.',
        149.90,
        'Audio',
        18,
        '2026-09-06 15:20:00'
    ),
    (
        'Webcam Full HD',
        'Webcam Full HD 1080p con microfono integrato.',
        69.90,
        'Accessori',
        27,
        '2026-09-07 10:10:00'
    ),
    (
        'Microfono USB Studio',
        'Microfono USB a condensatore per streaming, podcast e
        videoconferenze.',
        119.00,
        'Audio',
        14,
        '2026-09-08 13:40:00'
    ),
    (
        'SSD Esterno 1TB',
        'Unità SSD esterna USB 3.2 con capacità di 1 TB.',
        109.90,
        'Storage',
        31,
        '2026-09-09 16:00:00'
    ),
    (
        'Hard Disk Esterno 4TB',
        'Hard disk esterno USB con capacità di 4 TB.',
        129.90,
        'Storage',
        19,
        '2026-09-10 09:30:00'
    ),
    (
        'Tablet 11',
        'Tablet da 11 pollici con 128 GB di memoria interna.',
        399.90,
        'Mobile',
        16,
        '2026-09-11 11:00:00'
    )

```

```

),
(
    'Smartphone X',
    'Smartphone con display OLED, 256 GB di memoria e fotocamera
    avanzata.',
    799.90,
    'Mobile',
    20,
    '2026-09-12 12:15:00'
),
(
    'Smartwatch Active',
    'Smartwatch per monitoraggio attività fisica e notifiche.',
    249.90,
    'Wearable',
    25,
    '2026-09-13 14:30:00'
),
(
    'Router WiFi 6',
    'Router wireless WiFi 6 per reti domestiche e piccoli uffici.',
    159.90,
    'Networking',
    17,
    '2026-09-14 10:00:00'
),
(
    'Switch Gigabit 8 Porte',
    'Switch Ethernet Gigabit a 8 porte.',
    49.90,
    'Networking',
    40,
    '2026-09-15 15:45:00'
),
(
    'Stampante Laser',
    'Stampante laser monocromatica per uso professionale.',
    219.90,
    'Ufficio',
    9,
    '2026-09-16 09:20:00'
),
(
    'Sedia Ergonomica',
    'Sedia da ufficio ergonomica con supporto lombare regolabile.',
    329.90,
    'Ufficio',
    11,
    '2026-09-17 13:10:00'
),
(
    'Scrivania Regolabile',
    'Scrivania elettrica regolabile in altezza.',
    549.90,

```

```

        'Ufficio',
        7,
        '2026-09-18 10:30:00'
    ),
    (
        'Power Bank 20000mAh',
        'Batteria portatile ad alta capacità con ricarica rapida.',
        39.90,
        'Accessori',
        55,
        '2026-09-19 16:20:00'
    ),
    (
        'Hub USB-C',
        'Hub USB-C multiporta con HDMI, USB 3.0 e lettore SD.',
        74.90,
        'Accessori',
        29,
        '2026-09-20 11:45:00'
    );

```

I dati permettono di verificare facilmente:

- prodotti appartenenti alla stessa categoria;
- ricerca per nome;
- ordinamento per prezzo;
- prodotti con quantità diverse;
- prodotti con prezzi diversi;
- interrogazioni REST con risultati multipli.

Producer Application

La prima applicazione deve essere denominata, ad esempio:

```
product-api
```

e deve essere eseguita sulla porta:

```
8081
```

Configurazione indicativa:

```
spring.application.name=product-api
```

```

server.port=8081

spring.datasource.url=jdbc:mysql://localhost:3306/esercitazione_api
spring.datasource.username=root
spring.datasource.password=PASSWORD

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

```

La password deve essere modificata in base alla configurazione locale di MySQL.

Struttura della Producer Application

Si consiglia una struttura organizzata secondo il seguente modello:

```

src/main/java
├── it.esercitazione.api
│   ├── ProductApiApplication.java
│   ├── controller
│   │   └── ProdottoController.java
│   ├── service
│   │   └── ProdottoService.java
│   ├── repository
│   │   └── ProdottoRepository.java
│   ├── model
│   │   └── Prodotto.java
│   ├── dto
│   │   ├── ProdottoRequest.java
│   │   └── ProdottoResponse.java
│   └── exception
│       ├── ProdottoNotFoundException.java
│       └── GlobalExceptionHandler.java

```

La suddivisione in livelli deve essere rispettata:

```

Controller
  ↓
Service
  ↓
Repository

```

↓
Database

Il controller non deve contenere direttamente la logica di accesso al database.

Entity Prodotto

Creare una classe `Prodotto` associata alla tabella `prodotti`.

La classe deve contenere almeno:

```
id
nome
descrizione
prezzo
categoria
quantita
dataCreazione
```

L'identificativo deve essere generato automaticamente.

Il campo `prezzo` deve essere rappresentato evitando l'uso di `double` per la gestione del valore monetario. È consigliato utilizzare:

```
BigDecimal
```

La data può essere rappresentata utilizzando una classe della Java Date/Time API, ad esempio:

```
LocalDateTime
```

Repository

Creare un'interfaccia:

```
ProdottoRepository
```

che estenda:

```
JpaRepository<Prodotto, Long>
```

Il repository dovrà permettere di effettuare le operazioni CRUD e, successivamente, le ricerche richieste dal livello intermedio.

Service

Creare un servizio:

```
ProdottoService
```

che gestisca la logica applicativa.

Il service deve prevedere almeno i seguenti metodi:

```
creaProdotto()  
trovaTutti()  
trovaPerId()  
aggiornaProdotto()  
eliminaProdotto()
```

A questi dovranno essere aggiunti i metodi necessari per:

```
trovaPerCategoria()  
cercaPerNome()  
ordinaPerPrezzo()
```

REST API

La base URL della Producer Application deve essere:

```
http://localhost:8081/api/prodotti
```

Creazione prodotto

```
POST /api/prodotti
```

Esempio:

```
{
  "nome": "Notebook Gaming",
  "descrizione": "Notebook per gaming e sviluppo software",
  "prezzo": 1599.90,
  "categoria": "Informatica",
  "quantita": 10
}
```

Risposta prevista:

```
201 Created
```

Lettura di tutti i prodotti

```
GET /api/prodotti
```

Esempio di risposta:

```
[
  {
    "id": 1,
    "nome": "Laptop Pro 15",
    "descrizione": "Notebook professionale...",
    "prezzo": 1299.90,
    "categoria": "Informatica",
    "quantita": 15,
    "dataCreazione": "2026-09-01T09:00:00"
  }
]
```

Lettura di un singolo prodotto

```
GET /api/prodotti/{id}
```

Esempio:

```
GET /api/prodotti/5
```

Se il prodotto esiste:

```
200 OK
```

Se il prodotto non esiste:

```
404 Not Found
```

Aggiornamento prodotto

```
PUT /api/prodotti/{id}
```

Esempio:

```
PUT /api/prodotti/5
```

Body:

```
{
  "nome": "Mouse Wireless Pro X",
  "descrizione": "Mouse wireless professionale",
  "prezzo": 69.90,
  "categoria": "Accessori",
  "quantita": 42
}
```

Risposta:

```
200 OK
```

Eliminazione prodotto

```
DELETE /api/prodotti/{id}
```

Esempio:

```
DELETE /api/prodotti/5
```

Risposta:

```
204 No Content
```

Se il prodotto non esiste:

```
404 Not Found
```

Ricerca per categoria

Implementare la possibilità di ottenere tutti i prodotti appartenenti a una determinata categoria.

Endpoint:

```
GET /api/prodotti?categoria=Informatica
```

La ricerca deve essere effettuata dal database attraverso Spring Data JPA.

Esempio di risultato:

```
Laptop Pro 15  
Laptop Air 13  
Monitor 27 4K
```

Ricerca per nome

Implementare una ricerca testuale sul nome del prodotto.

Esempio:

```
GET /api/prodotti?nome=laptop
```

La ricerca dovrebbe consentire di trovare anche nomi che contengono il termine cercato.

Ad esempio:

```
laptop
```

deve trovare:

```
Laptop Pro 15  
Laptop Air 13
```

La ricerca non dovrebbe dipendere dalla presenza di maiuscole/minuscole.

Ordinamento per prezzo

Implementare l'ordinamento dei prodotti per prezzo.

Esempio:

```
GET /api/prodotti?sort=prezzo
```

Ordinamento crescente.

Prevedere eventualmente anche:

```
GET /api/prodotti?sort=prezzo&direction=desc
```

per ottenere l'ordinamento decrescente.

Gestione delle combinazioni

Gli endpoint dovrebbero essere progettati in modo da poter combinare i criteri.

Ad esempio:

```
GET /api/prodotti?categoria=Accessori&sort=prezzo&direction=asc
```

oppure:

```
GET /api/prodotti?nome=pro&sort=prezzo&direction=desc
```

Gestione degli errori

La Producer Application deve gestire correttamente gli errori.

In particolare:

Prodotto inesistente

```
GET /api/prodotti/9999
```

deve produrre:

```
404 Not Found
```

con una risposta JSON simile a:

```
{
  "status": 404,
  "message": "Prodotto non trovato",
  "timestamp": "2026-09-28T18:30:00"
}
```

Dati non validi

Una richiesta contenente dati non validi deve produrre:

```
400 Bad Request
```

La gestione degli errori deve essere centralizzata, ad esempio attraverso:

```
@RestControllerAdvice
```

Consumer Application

La seconda applicazione deve essere denominata, ad esempio:

```
product-client
```

e deve essere eseguita sulla porta:

```
8082
```

La Consumer Application **non deve utilizzare direttamente JPA per accedere alla tabella `prodotti`**.

Il flusso deve essere:

```
Browser
  ↓
Consumer Application
  ↓
HTTP
  ↓
Producer REST API
  ↓
Service
  ↓
Repository
  ↓
MySQL
```

Configurazione Consumer

La Consumer Application deve utilizzare:

```
spring.application.name=product-client  
  
server.port=8082
```

L'indirizzo della Producer API dovrebbe essere configurabile attraverso una property:

```
api.base-url=http://localhost:8081/api
```

In questo modo l'URL non viene scritto direttamente all'interno del codice Java.

Client HTTP

Creare un componente responsabile della comunicazione con la Producer Application.

Ad esempio:

```
ProdottoApiClient
```

Il client deve effettuare almeno le seguenti chiamate:

```
GET /api/prodotti  
GET /api/prodotti/{id}
```

Il client deve convertire il JSON ricevuto dalla Producer Application in oggetti Java.

DTO

La Consumer Application deve utilizzare un DTO dedicato.

Esempio concettuale:

```
ProdottoDTO  
  
id  
nome  
descrizione  
prezzo  
categoria
```

```
quantita  
dataCreazione
```

Il DTO non deve essere necessariamente identico all'Entity utilizzata nella Producer Application.

L'obiettivo è mantenere separate:

```
Database Entity  
    ≠  
API DTO  
    ≠  
Client Model
```

Interfaccia Web

La Consumer Application deve utilizzare Thymeleaf per realizzare una pagina:

```
/prodotti
```

La pagina deve visualizzare i prodotti in una tabella.

Esempio:

ID	Nome	Categoria	Prezzo	Quantità
1	Laptop Pro 15	Informatica	€ 1.299,90	15
2	Laptop Air 13	Informatica	€ 899,00	22
3	Monitor 27 4K	Informatica	€ 449,90	12

La pagina deve essere generata utilizzando:

```
Controller  
    ↓  
API Client  
    ↓  
REST API  
    ↓  
JSON  
    ↓  
DTO  
    ↓  
Thymeleaf
```

Dettaglio prodotto

Implementare una pagina:

```
/prodotti/{id}
```

che visualizzi il dettaglio del prodotto selezionato.

Ad esempio:

```
-----
Laptop Pro 15

Categoria: Informatica

Prezzo: € 1.299,90

Quantità disponibile: 15

Descrizione:
Notebook professionale con processore
Intel Core i7, 16 GB di RAM e SSD da 512 GB.
-----
```

Ricerca dalla Consumer Application

La pagina web deve permettere all'utente di effettuare almeno una ricerca per:

```
Nome
```

e/o:

```
Categoria
```

Esempio:

```
+-----+
| Cerca prodotto: [ laptop      ] |
| Categoria:      [ Tutte ▼   ] |
```

```
|                                     |  
|               [ CERCA ]           |  
+-----+  
+-----+
```

La Consumer Application dovrà costruire la richiesta HTTP verso la Producer API.

Ad esempio:

```
GET http://localhost:8081/api/prodotti?nome=laptop
```

Livello base

Devono essere implementate tutte le seguenti funzionalità:

- creare prodotto;
- leggere tutti i prodotti;
- leggere il dettaglio di un prodotto;
- aggiornare un prodotto;
- eliminare un prodotto.

Le operazioni CRUD devono essere disponibili nella Producer Application.

Livello intermedio

Implementare:

- filtro per categoria;
 - ricerca per nome;
 - ordinamento per prezzo;
 - combinazione dei parametri di ricerca;
 - gestione dei parametri attraverso `@RequestParam`;
 - query Spring Data JPA dedicate.
-

Livello avanzato

Implementare nella Consumer Application:

- client HTTP;
- comunicazione con la REST API;
- DTO;
- pagina HTML con Thymeleaf;
- tabella prodotti;

- pagina dettaglio;
- ricerca;
- gestione degli errori provenienti dalla REST API;
- messaggio appropriato quando la Producer Application non è raggiungibile.

Esempio:

```
Impossibile recuperare i prodotti.  
  
Il servizio API non è attualmente disponibile.
```

Livello esperto

Implementare funzionalità aggiuntive.

Autenticazione semplice

Proteggere alcune API attraverso una semplice autenticazione.

Ad esempio:

```
GET    /api/prodotti      → pubblico  
GET    /api/prodotti/{id} → pubblico  
  
POST   /api/prodotti      → autenticazione richiesta  
PUT    /api/prodotti/{id} → autenticazione richiesta  
DELETE /api/prodotti/{id} → autenticazione richiesta
```

È possibile utilizzare:

```
Spring Security
```

Cache locale

Implementare una cache nella Consumer Application.

Lo scopo è evitare di effettuare una nuova chiamata HTTP alla Producer Application quando i dati sono già disponibili localmente e non sono scaduti.

È possibile utilizzare, ad esempio:

```
Spring Cache
```

con una cache in memoria.

DTO dedicati

Separare chiaramente:

```
Entity
  ↓
API Request DTO
  ↓
API Response DTO
```

Ad esempio:

```
ProdottoRequest
ProdottoResponse
```

invece di esporre direttamente l'Entity JPA attraverso il controller.

Test della REST API

La Producer Application deve essere testata utilizzando uno strumento come:

- Postman;
- Insomnia;
- curl;
- REST Client di IntelliJ IDEA;
- REST Client di Visual Studio Code.

Devono essere verificate almeno le seguenti richieste.

GET

```
GET http://localhost:8081/api/prodotti
```

GET dettaglio

```
GET http://localhost:8081/api/prodotti/1
```

GET categoria

```
GET http://localhost:8081/api/prodotti?categoria=Informatica
```

GET ricerca

```
GET http://localhost:8081/api/prodotti?nome=laptop
```

GET ordinamento

```
GET http://localhost:8081/api/prodotti?sort=prezzo&direction=desc
```

POST

```
POST http://localhost:8081/api/prodotti
```

con:

```
{
  "nome": "Notebook Gaming",
  "descrizione": "Notebook ad alte prestazioni",
  "prezzo": 1599.90,
  "categoria": "Informatica",
  "quantita": 8
}
```

PUT

```
PUT http://localhost:8081/api/prodotti/1
```

DELETE

```
DELETE http://localhost:8081/api/prodotti/20
```

Flusso di lavoro

1. Avvio MySQL

Avviare il server MySQL.

2. Creazione database

Creare:

```
esercitazione_api
```

3. Caricamento dati

Eseguire lo script SQL fornito nell'esercitazione.

Verificare:

```
SELECT * FROM prodotti;
```

Dovrebbero essere presenti almeno 20 prodotti.

4. Avvio Producer

Avviare la Producer Application sulla porta:

```
8081
```

Verificare:

```
http://localhost:8081/api/prodotti
```

5. Avvio Consumer

Avviare la Consumer Application sulla porta:

```
8082
```

Verificare:

```
http://localhost:8082/prodotti
```

6. Verifica comunicazione

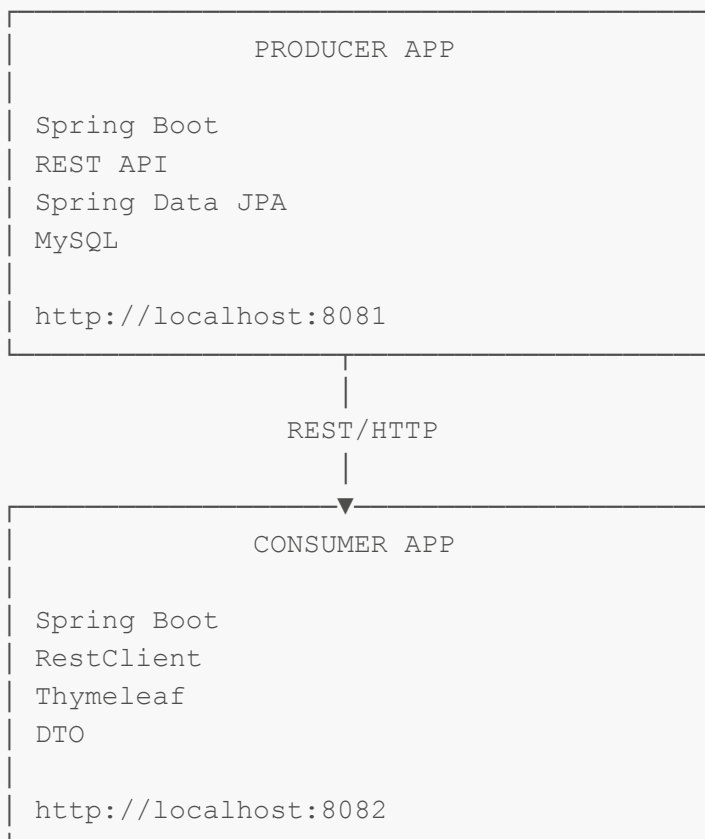
Il browser deve comunicare con:

```
Consumer
  ↓
Producer
  ↓
MySQL
```

La Consumer Application **non deve collegarsi direttamente a MySQL**.

Risultato finale

Al termine dell'esercitazione devono essere disponibili due applicazioni:



La REST API deve consentire la gestione completa dei prodotti, mentre la Consumer Application deve utilizzare esclusivamente le API per recuperare e visualizzare i dati.

Consegna

Il progetto finale deve contenere:

```
producer/  
├── pom.xml  
├── src/  
└── README.md  
  
consumer/  
├── pom.xml  
├── src/  
└── README.md
```

Il `README.md` deve contenere almeno:

- descrizione del progetto;
- tecnologie utilizzate;
- requisiti necessari;
- configurazione MySQL;
- configurazione della Producer;
- configurazione della Consumer;
- istruzioni per l'avvio;
- elenco delle REST API disponibili;
- eventuali funzionalità aggiuntive implementate.